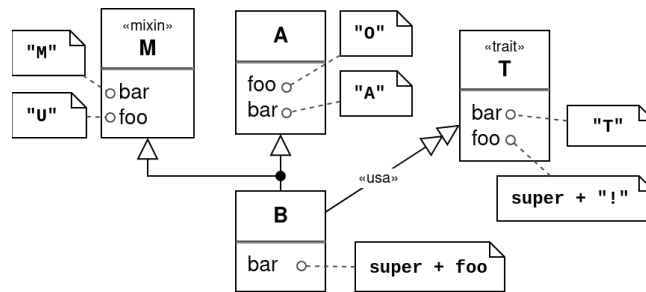


Modelo de primer parcial

El parcial se desarrolla en papel, y es a carpeta cerrada. La totalidad de las preguntas obligatorias suman 10 puntos. En este caso, incluimos también preguntas bonus, como ayuda para sumar puntaje extra. Algunas preguntas bonus pueden requerir conocimiento por fuera de las lecturas obligatorias (por ejemplo, de lecturas opcionales o de otros lenguajes que no hemos visto estrictamente dentro de la materia).

Preguntas obligatorias

1. Dado el siguiente modelo de un programa en Ruby que usa mixins y traits:



¿Qué retorna la colaboración B.new.bar? (¿O qué excepción levanta, en caso de fallar?). Explicá brevemente el proceso que se sigue para llegar al resultado final.

2. En Ruby, los métodos definidos con `def` no tienen acceso a su contexto léxico. Por ejemplo:

```
x = 1
def m1
  # Acá no existe x!
end
```

Sin embargo, podríamos pensar en un método (instancia de `Method`) como una especie de *closure*. ¿Por qué? ¿Cómo relacionarías el concepto de *closure* y el de *method* en Ruby? ¿Qué similitudes hay?

3. Alguien afirma lo siguiente:

Los programas en mi lenguaje de programación se expresan por medio de archivos de texto. Desde un programa se pueden leer y escribir archivos, usando la clase `File`. Esto incluye a los archivos que representan al programa en sí mismo, y por lo tanto el sistema es reflexivo.

¿Es suficiente lo que dice para concluir que el sistema es reflexivo? ¿Por qué, o por qué no?

4. En Ruby, el mensaje `send(selector, *argumentos)` permite enviar un mensaje y que se ejecuten tanto métodos públicos como privados. Además, se pueden leer y modificar las variables de instancia de un objeto usando los mensajes `instance_variable_get(identificador)` e `instance_variable_set(identificador, valor)`. A partir de esto, alguien concluye:

Por lo tanto, en Ruby no hay encapsulamiento.

¿Estarías de acuerdo con eso? Si pensás que el concepto de encapsulamiento se mantiene, explicá por qué. En el caso contrario, planteá una alternativa de diseño que permita mantener el encapsulamiento sin perder las capacidades reflexivas.

Preguntas bonus

5. Dentro de la reflexión computacional, distinguimos a la *introspección* y la *intercesión*. Explicá, brevemente, en qué consiste cada una, dando ejemplos.
6. En Ruby un objeto puede reflexionar sobre sí mismo (por ejemplo, podemos preguntarle qué método va a usar para responder un mensaje mediante el mensaje `method(selector)`). El enfoque de los *mirrors* propone, en cambio, darle esas responsabilidades a otro objeto. Realizá un breve análisis de esa decisión de diseño. ¿Qué se gana y qué se pierde cuando las capacidades reflexivas viven en el mismo objeto que usamos para representar una cosa del dominio de interés vs. en otro objeto?
7. Hemos distinguido dos maneras de entender a la programación, en particular:
 - a. entendiendo a un programa como un artefacto lingüístico (un texto estático que describe un cómputo, como en Java y en la mayoría de los lenguajes convencionales), o
 - b. entendiéndolo como un proceso “vivo” (un sistema en ejecución que se modifica a sí mismo, donde no hay una separación tajante entre “el programa” y “su ejecución”, como en Smalltalk o Lisp).

Elegí la visión que más te guste y argumentá por qué te parece la más adecuada para entender qué es programar. Tu descripción tiene que mencionar al menos una cosa que la visión alternativa resuelva mejor (o de manera menos complicada) que la que elegiste. También es posible tomar una postura intermedia, siempre y cuando haya una buena justificación.

8. En JavaScript hay muchas complicaciones con respecto a cómo se liga `this`, que causan confusión entre las personas que programan en ese lenguaje. Relacioná este problema con la ausencia de la distinción que hace Ruby entre `Method` y `UnboundMethod`. ¿Podrían haberse evitado estas confusiones si JavaScript hubiese hecho esa distinción? Justificá tu respuesta.